분류번호 : 2025-2-4102-01

졸업작품 최종보고서

AI 기반 스마트 덕트 관리 로봇

지도교수 : 도 현 락 교수님

제 출 일 : 2025년 11월 3일

제 작 자 : 20101461 고 은 찬

20101531 윤 한 준

20101482 김 연 재

20100315 손 윤 석

서울과학기술대학교 전자공학과

목 차

┌ 요 약 i
│ 표 목 차 ii
│ 그림목차 ii
└ 사진목차 ii

Ⅰ. 개요 [목표설정] 1

- 작품의 목표
- 제작 배경, 목표설정 동기, 문제점, 해결방안
- 설계 진행일정 : 작품구상, 목표설정, 자료수집, 분석, 합성, 제작, 시험 및 평가 등의 일정 계획 기술
- 구현과정에서 예측되는 현실적 제약조건 기술 :
  ①산업표준 ②경제요건 ③환경 ④윤리 ⑤안전 ⑥사회 ⑦정치 ⑧실행가능성

Ⅱ. 시스템의 설계 [분석, 합성] 5

- 5대 설계 원칙 구현 : 구상설계(idea 도출), 기능설계, 다목적설계, 경제적설계, 미관적설계
- 설계절차 기술; ①요소설계(H/W+S/W) → ②종합설계 → ③FeedBack 설계
- 시스템의 구성 도면
- S/W algorithm

Ⅲ. 시스템의 제작 [제작] 10

- 1학기 : 기본제작
- 2학기 : 보충 제작 및 feedback 제작
- 사용부품 명세서 등

Ⅳ.시스템의 동작 검증 [시험, 성능평가] 27

- 제작과정의 주요 부품 또는 모듈의 정상동작 확인을 위한 시험 항목과 시험결과
- 완성된 전체 시스템의 목표 성능에 대한 평가항목 및 평가 결과

Ⅴ. 결론 [결과도출] 33

- 목표설정 시스템과 개발결과 시스템의 기능과 성능을 종합 비교하여 완성도 서술
- 완성된 작품의 주요 활용분야 및 기타 주요 검토사항 서술

Ⅵ. 참고 문헌 36

Ⅶ. 부 록 37

Ⅷ. 감사의 글 39

요 약

제 목 : AI 기반 스마트 덕트 관리 로봇

본 졸업작품 최종보고서 “AI 기반 스마트 덕트 관리 로봇“은 협소하고 비정형적인 덕트 환경에서 숙련된 작업자의 수동 점검 방식이 갖는 안전성, 효율성, 접근성 한계 극복을 목표로 AI, 로보틱스, IoT 융합 자율 주행 로봇 시스템을 성공적으로 개발・검증하였다.

핵심 하드웨어 플랫폼은 저프로파일 캐터필러 트랙과 3D 프린팅 설계를 통해 덕트 내 협소한 공간에서의 안정적 주행과 우수한 접지력 확보를 실현하였다. 소프트웨어는 ROS2 기반 Google Cartographer SLAM 알고리즘 & SLAM_Toolbox 알고리즘 및 Nav2 네비게이션 스택을 중심으로 실시간 지도 작성과 자율 탐색 기능을 완성하였고, 자율 주행의 경로 최적화와 장애물 회피를 성공적으로 구현하였다. AI 비전 기술로는 경량화된 YOLOv11 Nano 모델을 탑재한 Raspberry Pi 환경에서 케이블 단선 및 피복 손상 실시간 탐지를 가능케 하였으며, 탐지된 데이터를 MongoDB Atlas 클라우드 기반으로 체계적 저장 및 관리하여 예측 유지보수의 기반을 마련하였다.

원격 웹 관제 시스템은 Flask 웹 서버와 ROSBridge를 통해 통합 운영되며, 실시간 영상 스트리밍부터 센서 정보, 로봇 상태, AI 분석 결과까지 사용자 친화적 UI로 시각화함으로써 비전문가도 손쉽게 현장 상황을 모니터링하고 제어할 수 있도록 설계되었다.

반복적인 프로토타입 테스트와 피드백 과정을 통해, 발생한 캐터필러 바퀴 결합 불량 및 SLAM 맵핑 오류 문제를 구조적・소프트웨어적으로 개선하였고, SLAM 패키지 중에서도 신뢰성과 성능의 균형을 고려하여 상황에 따라 필요에 맞게 SLAM_Toolbox 및 Cartographer 알고리즘을 선택하여 성능 검증을 수행하였다.

산업 현장 자동화에서 요구되는 안전성, 신뢰성 및 운영 효율성을 체계적으로 충족함으로써, 이 시스템은 데이터센터, 통신 인프라, 플랜트, 스마트 건축물 등 다양한 분야에 적용 가능하며, 위험한 사람의 직접 점검을 대체하는 혁신적 유지보수 자동화 솔루션을 개발하고자 하였다. 보다 학술적으로, 본 연구는 SLAM, AI 비전, 자율주행 로봇 시스템의 융합과 클라우드 기반 데이터 관리 구조를 통합 적용함으로써, 비정형 산업 환경에서의 스마트 유지관리 기술 발전에 기여하고자 하였다.

요약하면, 본 프로젝트는 4차 산업혁명 핵심 기술들을 통합하고 공조하여 현장 작업자의 안전 강화 목표를 달성하고, 데이터 기반 예측 유지보수 및 원격 관제를 가능케 한 지능형 로봇 개발 사례로, 향후 스마트 팩토리 및 인프라 유지관리 자동화 분야의 연구개발 및 상용화에 방향성을 제시할 수 있다고 기대한다.

표 목 차

Table. 1-1. 프로젝트 진행일정..... 3
Table. 5-1. 목표 기능 대비 개발 완료 기능의 성능 및 진척도 종합 비교표..... 33

그 림 목 차

Fig.2-2. Cartographer SLAM 알고리즘 흐름도..... 8
Fig.2-3. YOLO AI 모델..... 8
Fig.3-1. YOLO 기반 케이블 손상 객체 탐지 알고리즘..... 11
Fig. 3-2. 센서 데이터 녹화 과정..... 13
Fig. 3-3. Cartographer SLAM을 이용한 창학관 회로설계연구실 내부 맵핑 결과.. 13
Fig. 3-4. Explorer(자율탐색 알고리즘) 코드 일부..... 20
Fig. 3-5. 수정한 파라미터(1)..... 20
Fig. 3-6. 수정한 파라미터(2)..... 21
Fig. 3-7. M4 Processor에서의 RTSP 실험결과..... 22

Fig. 3-8. 카메라 설정 및 영상 스트리밍 코드..... 23
Fig. 3-9. 영상프레임 생성 함수 코드..... 23
Fig. 3-10. gen_frames() 함수에서 생성된 프레임을 반환 코드..... 23
Fig. 3-11. 전체적인 소프트웨어 구조..... 26

사 진 목 차

Photo.3-1. TurtleBot3 Burger..... 10
Photo.3-2. SAHI 라이브러리 적용 후 케이블 손상체크 실행 시의 화면..... 11
Photo. 3-3. 초기 케이블 결함탐지 데이터셋1..... 12
Photo. 3-4. 초기 케이블 결함탐지 데이터셋2..... 12
Photo. 3-5. Rviz2를 통한 센서 데이터 시각화..... 12
Photo. 3-6. TurtleBot3 Burger..... 14
Photo. 3-7. TB3 캐터필러 트랙..... 14
Photo. 3-8. 캐터필러 구동부 적용을 위한 1차 플랫폼 개조 형상..... 14
Photo. 3-9. 지름 4.5cm인 바퀴(좌) / 새로 설계한 지름 9cm 바퀴(우)..... 15
Photo. 3-10. 기존 로봇 바퀴와 새롭게 설계한 바퀴 비교..... 15
Photo. 3-11. 캐터필러 트랙 맞물림 개선을 위한 바퀴 톱니(Sprocket) 형상 비교 (상: 1차 설계, 하: 2차 수정 모델)..... 16
Photo. 3-12. 2차 수정한 로봇 바퀴..... 16
Photo. 3-13. 2차 설계 바퀴와 1차 설계 바퀴의 캐터필러 맞물림 비교..... 17
Photo. 3-14. 최종 개조 완료한 로봇 실물..... 17
Photo. 3-15. 초기 확보한 일자형 덕트(DUCT)..... 18
Photo. 3-16. 2차 복합 테스트를 위해 구축된 H자형 모의 덕트 환경..... 19
Photo. 3-17. SLAM_ToolBox를 이용한 mapping 사진..... 19
Photo. 3-18. 추가적으로 학습시킨 데이터셋 및 라벨링..... 21
Photo. 3-19. 제작한 로봇 제어 웹 UI/UX 화면..... 22
Photo. 3-20. 웹 내의 수동조작 메뉴 화면..... 24
Photo. 3-21. 웹 상 케이블 객체 탐지 확인 기능..... 24
Photo. 3-22. Atlas DB에 저장된 손상 케이블 데이터..... 25
Photo. 3-23. 웹상에서 덕트 내부 mapping 지도 확인 25
Photo. 4-1. 2차 설계 바퀴와 1차 설계 바퀴의 캐터필러 맞물림 비교..... 27
Photo. 4-2. 바퀴 연결부의 반복 내구시험..... 27
Photo. 4-3. 웹소켓 연결된 모습..... 28
Photo. 4-4. 덕트 데모 테스트 환경..... 29
Photo. 4-5. Mapping 진행 중인 터틀봇..... 29
Photo. 4-6. 연구실 내부 mapping 결과..... 30
Photo. 4-7. 실시간 케이블 손상체크 추론 사진..... 30
Photo. 4-8. 추가적으로 학습시킨 데이터셋 사진..... 31
Photo. 4-9. SLAM_Toolbox & Explorer 자율탐색 알고리즘 기반 자율주행..... 32

Ⅰ. 개 요

1. 작품의 목표
본 작품의 최종 목표는 AI 기반 스마트 덕트 관리 로봇 프로토타입 개발이다. 이는 상업용 건물, 통신 시설, 데이터센터 등 인간의 접근이 물리적으로 어렵거나 안전상의 이유로 제한되는 덕트(Duct) 환경에 투입되어, 기존의 수동적·주기적 점검 방식을 대체하는 지능형 자동화 시스템을 구현하는 것을 목적으로 한다.

이를 달성하기 위한 세부 기술 목표는 다음과 같다.

첫째, 덕트 환경 주행에 최적화된 하드웨어 플랫폼을 구축한다. 표준 로봇 플랫폼(TurtleBot3)은 덕트와 같이 좁고 협소한 환경에서는 원활한 주행에 한계가 존재한다. 이러한 한계를 극복하기 위해, 3D 프린터를 활용하여 저프로파일(Low-profile)의 캐터필러(Caterpillar) 트랙 구동부를 설계 및 적용함으로써, 협소하고 비정형적인 덕트 내부에서의 안정적인 주행 성능과 접지력을 확보한다.

둘째, SLAM(Simultaneous Localization and Mapping) 기반의 자율주행 시스템을 구현한다. ROS2(Robot Operating System 2) 환경에서 360° LiDAR 센서(LDS-02)와 Cartographer SLAM 알고리즘을 활용하여, 미지의 덕트 환경 지도를 실시간으로 생성함과 동시에 로봇의 정확한 현재 위치를 추정한다.

셋째, AI 비전 기반의 실시간 결함 탐지 기능을 개발한다. Raspberry Pi의 제한된 연산 자원 내에서 효율적으로 동작하는 경량화된 객체 탐지 모델(YOLOv8n/YOLOv11n)을 탑재하여, 주행 중 포착되는 케이블의 손상, 단선, 피복 벗겨짐 등의 이상 징후를 실시간으로 식별한다.

넷째, 원격 관제 및 데이터 관리 시스템을 통합한다. Flask 기반 웹 서버와 ROSBRIDGE를 연동하여 관리자가 원격지에서 로봇이 생성한 지도(Map)와 현재 상태를 JSON 형식으로 시각화하여 모니터링할 수 있도록 한다. 또한, AI가 탐지한 이상 지점의 정보(사진, 위치, 시간)를 MongoDB Atlas 클라우드 데이터베이스에 전송 및 저장하여, 수집된 데이터를 체계적으로 관리하고 향후 예측 유지보수를 위한 기반을 마련한다.

2. 제작 배경 및 동기
현대 산업 사회에서 데이터센터, 통신실, 발전소 등의 전력 및 통신 인프라는 핵심 기반 시설로 기능한다. 이러한 시설의 케이블은 대부분 건물의 덕트 내부에 밀집되어 설치된다. 그러나 덕트 내부는 구조적으로 협소하고 복잡하며, 분진이나 유해 가스가 존재할 수 있어 작업자의 접근이 극히 제한되는 고위험 작업 환경이다.
현재 이러한 덕트 내 케이블 점검은 대부분 숙련된 작업자의 육안에 의존하는 수동 방식으로 이루어지고 있다. 이 방식은 작업자의 추락, 감전 등의 중대재해 발생 가능성을 상시 내포하고 있으며, 높은 인건비와 많은 시간이 소요된다. 또한 점검 주기가 길어 케이블의 과열, 단선, 피복 손상과 같은 잠재적 위험 요소를 실시간으로 감지하기 어렵다.

이는 설비 장애로 인한 다운타임(Downtime)을 넘어, 화재와 같은 치명적인 2차 사고로 이어져 막대한 경제적, 사회적 손실을 유발할 수 있다.
이에 본 프로젝트에서는 4차 산업혁명의 핵심 기술인 로보틱스와 인공지능(AI)을 융합하여, 이러한 고위험 수동 작업을 대체하고 실시간 예방 점검이 가능한 지능형 로봇 시스템의 필요성을 절감하여 본 작품을 기획하게 되었다.

3. 기존 기술의 한계 및 문제점
현재 데이터센터 및 산업 현장의 덕트 관리는 숙련된 작업자의 육안에 의존하는 수동 점검 방식에 머물러 있으며, 이는 다음과 같은 명확한 한계를 내포하고 있다.

(1) 인적 요인의 안전성 한계
덕트 내부는 고소(高所), 협소, 분진 등이 복합적으로 작용하는 고위험 환경이다. 점검 작업자의 현장 투입은 추락, 감전 등 중대산업재해의 발생 가능성에 상시 노출되는 근본적인 문제를 야기한다.

(2) 경제적 비효율성
덕트와 같이 좁고 비정형적인 공간내에서의 수동 점검은 고비용의 숙련된 인력을 요구하며 점검 가능한 물리적 범위가 한정적이고 프로세스 자체가 시간 소모적이다. 이는 높은 운영 비용(TCO) 대비 낮은 점검 효율로 직결되어 경제성을 저하한다.

(3) 물리적・구조적 접근 한계
사다리형 덕트나 천장 내부와 같이 구조적으로 복잡한 구간은 작업자의 물리적 접근 자체가 불가능하다. 이는 필연적으로 점검 사각지대를 발생시켜 잠재적 위험 요소를 방치하게 된다.

(4) 데이터 연속성 및 실시간성 부재
기존의 주기적 점검 방식은 단속적인 데이터 수집에 그친다는 한계점이 존재한다. 이로 인해 케이블의 과열, 피복 손상 등 돌발적인 이상 징후를 실시간으로 탐지하는 것이 불가능하며, 사후 대응적 유지보수에 머무를 수밖에 없다.

4. 문제점 해결방안

본 작품은 앞서 제기된 4가지 핵심 한계를 극복하기 위해, 다음과 같은 로보틱스 및 AI 기술을 적용하여 체계적인 해결방안을 제시한다.

(1) 위험 환경 작업의 무인화
작업자를 위험 요소로부터 원천적으로 분리하기 위해, 비정형 환경 주행에 최적화된 저프로파일 캐터필러 구동부를 탑재한 로봇을 투입한다. 이는 덕트(DUCT)내의 원활한 주행과 구조적 한계로 인한 점검 사각지대 문제를 동시에 해결한다.

(2) SLAM 기반 점검 프로세스 자동화
LiDAR 센서(LDS-02)와 ROS2 Cartographer 알고리즘을 기반으로, 로봇이 스스로 덕트 내부 환경을 맵핑(Mapping)하고 최적 경로로 자율주행 점검을 수행한다. 이는 점검 시간을 획기적으로 단축하고 장기적인 운영 비용을 최소화한다.

(3) AI 및 DB 연동을 통한 예방 정비 시스템 구축
YOLO(v8n/v11n) AI 비전 모델을 통해 실시간으로 케이블의 결함을 탐지한다.

탐지된 이상 징후는 즉시 위치, 시간, 사진 데이터와 함께 MongoDB Atlas 클라우드 DB로 전송·축적된다. 이는 관리자의 즉각적인 대응을 지원하며, 데이터 기반의 예측 정비를 가능하게 한다.

5. 프로젝트 진행일정

구분	추진내용	프로젝트 기간								
		2월	3월	4월	5월	6월	7월	8월	9월	10월
계획	프로젝트 목표 설정, 역할 분배, 개발 일정 수립									
분석	덕트 환경 조사, 요구사항 분석, 유사 기술 및 기존 로봇 사례 연구									
설계	하드웨어(로봇 플랫폼, 라스베리파이, 다양한 센서 등) 설계									
	소프트웨어(SLAM, LiDAR, IoT, ROS, 동키카 등) 설계									
개발	로봇 하드웨어 제작 및 부품 조립, 센서 및 모터 제어 프로그래밍 개발									
	SLAM 및 LiDAR 기반 자율주행 및 환경 인식 알고리즘 구현									
	IoT 연동을 통한 원격 모니터링 및 제어 시스템 개발									
테스트	로봇 주행 실험, 데이터 수집 및 성능 검증 및 문제점 개선									
종료	최종 보고서 작성, 논문 및 포스터 발표 준비, 프로젝트 결과 정리									

Table.1-1. 프로젝트 진행일정

6. 예측되는 현실적 제약조건

본 프로젝트는 학부 캡스톤디자인 과정의 일환으로 수행됨에 따라, 실제 산업 현장과는 상이한 다음과 같은 8가지의 현실적 제약조건 하에서 기획 및 개발이 진행되었다.

(1) 산업 표준

본 프로젝트는 로봇 개발의 사실상 표준인 ROS2를 채택하였다. 이는 방대한 오픈소스 생태계와 높은 확장성을 제공하지만 ROS2는 표준임에도 불구하고 상용솔루션이 아닌 오픈소스라는 태생적 한계를 가진다. 이는 즉각적인 기술 지원 부재, 예측 불가능한 버그, 그리고 핵심 통신 프로토콜(DDS)의 보안 취약성 등등 향후 상용화 시 안정성과 유지보수 측면에서 해결하기 어려운 제약으로 작용한다

(2) 경제 요건

프로토타입 개발임에도 불구하고, 오픈소스 플랫폼인 터틀봇3(TurtleBot3)의 기본 구매 단가(약 80-90만 원)와 핵심 센서인 LiDAR(LDS-02)는 학술 연구 예산 내에서 상당한 비용 부담을 야기한다. 이는 로봇의 대량 생산이나 다중 로봇 시스템으로의 확장을 시도할 때, 초기 도입 비용(측면에서 극복하기 어려운 경제적 장벽으로 작용할 수 있다.

(3) 환경 요건

본 로봇은 '협소한 덕트'라는 극한의 환경적 제약을 전제로 설계되었다.
이 환경에 적응하기 위해 채택한 저프로파일 캐터필러(Low-profile Caterpillar) 설계는, 필연적으로 기동성 저하라는 물리적 한계를 야기한다. 특히, 제자리 회전시 높은 마찰 저항으로 인해 정밀한 조향이 어렵고 모터에 가해지는 부하가 크며 배터리 소모가 크다는 것을 발견하였다. 즉, 덕트 주행을 위한 안정성과 기동성(정밀 회전)은 양립하기 어렵다는 트레이드오프가 존재한다.

(4) 윤리

본 로봇은 케이블 결함 탐지를 위해 Raspberry PI 카메라를 사용한다. 로봇의 주된 임무는 설비 점검이지만, 이 과정에서 덕트 주변 작업 공간이나 관리자의 모습 등 의도치 않은 시각적 데이터를 수집할 가능성을 배제할 수 없다.
수집된 데이터의 소유권, 접근 권한, 프라이버시 침해 가능성에 대한 윤리적 문제는, 기술적으로 완벽히 통제하기 어렵다.

(5) 안전

로봇의 핵심 센서는 LDS-02 2D LiDAR 센서다. 이는 수평 2D 평면상의 장애물은 정확히 감지하나, 덕트 천장에서 늘어뜨려진 케이블이나 바닥의 작은 단차(턱)와 같은 수직적 장애물은 인식하지 못하는 본질적인 한계가 명확하다. 3D LiDAR로 교체하지 않는 한, 2D SLAM 기반 시스템은 상시 잠재적인 충돌 및 전복 위험을 내포한다.

(6) 사회

본 프로젝트의 목적은 작업자의 안전 확보이나, 장기적인 관점에서 이러한 자동화 로봇의 도입은 기존 유지보수 인력의 고용 감소 또는 직무 대체라는 사회적 이슈를 야기할 수 있다. 기술 발전이 노동 시장 구조에 미치는 영향은 본 캡스톤디자인의 범위를 넘어서는, 해결하기 어려운 거시적 제약이다.

Ⅱ. 시스템의 설계[분석, 합성]

1. 5대 설계 원칙 구현

1) 구상설계 (Idea Derivation)
본 시스템의 구상은 덕트(Duct) 내부의 수동 점검 방식이 내포한 안전성, 효율성, 접근성의 한계를 인식하는 것에서 출발하였다. 아이디어의 핵심은 단순한 원격 조종(RC) 로봇이 아닌, SLAM(동시적 위치추정 및 지도작성) 기반의 자율주행 기술과 AI 기반의 실시간 결함 탐지 기술을 융합하는 데 있다. 이를 통해 작업자의 개입을 최소화하고 위험 환경을 완전히 무인화하는 '지능형 관리 로봇'이라는 구체적인 아이디어를 도출하였다.

2) 기능설계
(a) 항법(Navigation) 기능: ROS2(Cartographer/Nav2) 와 360° LiDAR 센서 를 기반으로 한 SLAM 및 자율주행 기능을 설계하였다.

(b) 결함 탐지(Defect Detection) 기능: Raspberry Pi 와 Raspberry PI 카메라, 그리고 경량화된 YOLO 객체 탐지 모델을 연동하여 실시간 케이블 손상을 식별하도록 설계하였다.

(c) 데이터 관리(Data Management) 기능: 탐지된 결함 정보를(결함 정보를 발견한 당시 로봇 위치, 시간, 결함사진 등등) MongoDB Atlas 클라우드 DB에 저장하여 데이터의 영속성을 확보하도록 설계하였다.

(d) 모니터링(Monitoring) 기능: Flask 웹 서버와 ROSBRIDGE를 미들웨어로 사용하여, ROS2 환경에서 생성된 지도(Map) 데이터를 JSON 형식으로 변환, 웹 UI에서 시각화할 수 있도록 설계하였다.

3) 다목적설계
본 시스템은 모듈화(Modular) 설계를 채택하여 높은 확장성과 다목적성을 지닌다. AI 모델은 케이블 손상 외에도 가스 누출, 배관 부식 등 다른 데이터셋으로 재학습하여 교체 탑재가 가능하다. 또한, 핵심이 되는 소프트웨어 아키텍처(ROS2&WEB)는 본 로봇 플랫폼(H/W)과 분리되어 있어, 향후 터널, 교량 하부 등 타 비정형 환경 점검용 로봇에도 재사용이 가능한 다목적 지능형 점검 플랫폼으로의 확장을 고려하였다.

4) 경제적설계
본 설계는 한정된 캡스톤디자인 예산 내에서 가격 대비 성능비를 극대화하는 것을 목표로 하였다. 고가의 산업용 로봇 대신, 비교적 저렴한 오픈소스 하드웨어(TurtleBot3, Raspberry Pi 4)를 기본 플랫폼으로 채택하여 초기 도입 비용을 최소화했다. 또한, 캐터필러 플랫폼을 신규 구매하는 대신, 교내 3D 프린터를 활용하여 기존 플랫폼에 맞는 커스텀 구동 바퀴를 직접 설계 및 제작함으로써, 최소한의 비용으로 목표 기능(덕트 주행)을 구현하는 경제적 설계를 수행하였다.

5) 미관적설계
본 로봇의 설계는 순수한 심미성보다는 형태는 기능을 따른다는 원칙을 적용하였다. 덕트라는 극한의 협소 환경을 통과하기 위해, 불필요한 장식을 배제하고 모든 컴포넌트(LiDAR, 카메라, RPi 보드)를 저프로파일 형태로 수평 배치하였다. 이는 로봇의 전체 높이를 낮추고 무게 중심을 안정화하여 주행 성능을 극대화하기 위한 기능적 목적이 미관적 형태로 구현하였다.

2. 설계 절차
2.1)요소설계 (Element Design: H/W + S/W)
- H/W 요소 : Turtlebot3 플랫폼을 기반으로 메인 제어기(Raspberry Pi 4) , 모터 제어기(OpenCR Board) , 주 센서(LDS-02 LiDAR) , 비전 센서(AI camera) 등 핵심 부품을 선정하였다. 또한, 덕트 주행을 위한 핵심 기구부인 캐터필러 트랙 맞춤형 구동 바퀴를 Fusion 360으로 3D 모델링하여 개별 요소를 설계하였다.

- S/W 요소 : OS(ROS2) , SLAM(Cartographer) , 자율 탐색(Exploration Pkg), 객체 탐지(YOLO) , 웹 서버(Flask) , DB(MongoDB) 등 개별 소프트웨어 스택과 알고리즘을 선정하고 모듈별 아키텍처를 설계하였다.

2.2) 종합설계
H/W와 S/W는 ROS2를 기반으로 연동되며, Raspberry Pi(SBC)가 LiDAR, Raspberry PI 카메라의 데이터를 취합하고 OpenCR(MCU)로 모터 제어 명령을 전달하도록 종합하였다.
S/W 간의 통합은 ROSBRIDGE(port:9090)와 Flask Websocket(port:9091)을 핵심 미들웨어로 사용하여, 로봇 제어 영역(ROS2)과 사용자 인터페이스 영역(Flask Web Server)이라는 이기종 시스템 간의 데이터 흐름을 정의하였다.

2.3) FeedBack 설계
캡스톤디자인(1)의 교과목을 통해 초기 기본적인 자율주행 로봇 프로토타입을 제작해놓고 테스트 과정에서 발생하는 문제점을 다시 반복적으로 개선을 진행하였다.

(H/W Feedback) : 3D프린터를 통해 1차 출력된 바퀴로 주행 테스트를 해 본 결과, 바퀴 톱니와 캐터필러 트랙 간의 불일치로 인한 트랙 이탈 및 조향 불가라는 치명적인 피드백을 받았다. 이는 즉시 요소설계 단계로 환류되어, 트랙의 곡률에 정확히 맞추어 2차 바퀴 모델의 재설계를 진행하였다.

(S/W Feedback) : SLAM 맵핑(시험) 성공 후 지면과 덕트 공간내에서 자율주행을 테스트를 한 결과, 로봇이 맵 상의 벽을 인지함에도 불구하고 지속적으로 충돌하는 피드백을 수신하였다. 이는 종합설계 단계로 환류되어, Nav2 프로그램의 Costmap 및 Controller 관련 파라미터를 튜닝하는 재설계 작업을 진행하였다.

3. 시스템의 구성 도면

Fig. 2-1. AI 기반 스마트 덕트 관리 로봇 시스템 구성도

[Fig. 2-1]의 경우 본 AI 기반 스마트 덕트 관리 로봇의 시스템 기능 구성도를 개략적으로 도식화한 것이다. 본 시스템은 크게 사용자(USER)가 상호작용하는 웹 인터페이스(WEB)와, 실제 덕트 환경에서 임무를 수행하는 로봇 플랫폼으로 이원화된다. 설계한 시스템 기능 구조도에 대한 설명은 다음과 같다.

(1) 웹 인터페이스(WEB) 및 사용자(USER)
사용자는 PCv또는 모바일 디바이스를 통해 원격 웹 대시보드에 접속하며, 이는 시스템의 HCI(Human-Computer Interaction) 관문 역할을 수행한다. 사용자는 '주행로봇 제어 또는 자율주행 기능 on'과 같은 명령을 웹 서버로 송신한다. 동시에, 로봇 플랫폼의 AI 카메라 및 센서로부터 실시간 영상 및 정보를 피드백(Feedback)받아 원격 모니터링을 수행이 가능하다.

(2) 로봇 플랫폼
로봇 플랫폼은 제어부, 구동부, 기구부로 정밀하게 설계되었다.
(a) 제어부 : Raspberry Pi 4가 메인 SBC(Single-Board Computer)로서 시스템의 연산 처리를 총괄한다. 이는 웹 서버와의 양방향 통신, LiDAR(도면 외) 및 환경 센서(Sensor, 예: 온습도) 데이터 취합, 그리고 Raspberry PI 카메라로부터 얻은 영상의 스트림 처리를 담당한다.

(b) 구동부 : 로봇의 실제 구동은 모터 제어용 MCU인 OpenCR Board가 담당한다. Raspberry Pi로부터 ROS Topic 형태의 제어 명령을 수신하여 두 구동축의 모터를 정밀하게 제어한다.

(c) 기구부 : 덕트(Duct) 환경의 비정형적이고 협소한 노면 주행을 위해, 표준 Turtle Bot3 플랫폼의 휠(Wheel)을 제거하였다. 대신, 3D 프린터로 자체 설계한 탱크형 바퀴(캐터필러)를 장착하여 로봇의 접지력과 장애물 극복 성능을 확보하였다.

4. S/W algorithm

본 시스템의 핵심 기능은 두 가지 주요 S/W 알고리즘을 기반으로 구현되었다.

4.1) Cartographer SLAM

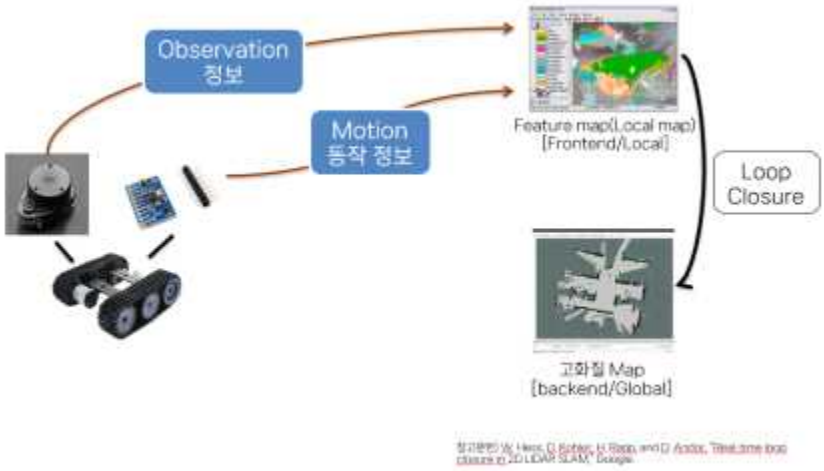

Fig. 2-2. Cartographer SLAM 알고리즘 흐름도

본 프로젝트의 동시 위치추정 및 지도작성(SLAM) 알고리즘은 Google에서 개발한 오픈소스 그래프 기반 SLAM 라이브러리인 Cartographer를 채택하였습니다.

Cartographer SLAM은 여러 플랫폼과 센서 구성에 걸쳐 2D공간과 3D공간에서 실시간 동시 위치 추정 및 지도 작성(Mapping)을 제공하는 시스템이다. 로봇은 해당 시스템을 활용하여 LDS-02 LiDAR 센서 및 IMU 센서를 통해 확보한 데이터를 기반으로 주행을 하며 지도를 작성한다.

본 로봇 시스템에서는 LDS-02 2D LiDAR 센서로부터 수신되는 스캔 데이터와 IMU(관성 측정 장치)의 데이터를 실시간으로 융합한다. 이 융합된 데이터를 기반으로, 로봇은 주행하며 일련의 '서브맵'을 생성한다. 이후, 누적된 서브맵 간의 루프 클로저 최적화를 비동기적으로 수행함으로써, 주행 누적 오차를 효과적으로 보정하고 전역적으로 일관성 있는 2D 점유 격자 지도를 작성한다.

4.2) AI 기반 실시간 결함 탐지(YOLO Algorithm)

Fig. 2-3. YOLO AI 모델

본 프로젝트의 실시간 케이블 결함 탐지 기능은 YOLO(You Only Look Once) 알고리즘을 기반으로 구현한다. YOLO는 객체 탐지(Object Detection)를 위한 단일 단계(one-stage) 딥러닝 모델이다.

해당 알고리즘은 전체 이미지를 단일 전방향 신경망(feed-forward neural network)에 한 번만 통과시켜, 객체의 경계 상자(bounding box)와 클래스 확률(class probability)을 동시에 예측한다.

이는 R-CNN 계열로 대표되는 2-stage 방식이 영역 후보(region proposal)를 먼저 생성한 후, 각 후보 영역을 개별적으로 분류하는 다단계 파이프라인을 따르는 것과 근본적인 차이를 보인다. YOLO는 객체 탐지 문제를 단일 회귀 문제(a single regression problem)로 재정의하여, 통합된 단일 신경망이 전체 이미지로부터 직접 경계 상자와 클래스 확률을 도출한다.

이러한 단일 단계 파이프라인을 가진 아키텍처는 복잡한 연산 파이프라인을 제거함으로써, 기존 2-stage 방식 대비 월등히 빠른 추론 속도를 보장한다. 이는 본 프로젝트의 목표인 Raspberry Pi와 같은 저전력 임베디드 시스템 환경에서 실시간(real-time) 결함 탐지 기능을 구현하는 데 가장 적합한 알고리즘이라고 생각했기에 채택하였다.

4.3) 자율 탐색 알고리즘
본 시스템은 작업자의 수동 조작 없이 미지의 덕트 환경을 스스로 탐색하며 지도를 완성하기 위해, SLAM 및 내비게이션 스택과 연동되는 상위 레벨의 자율 탐색(Autonomous Exploration) 알고리즘을 구현한다.

해당 알고리즘의 핵심은 Frontier-based Exploration (경계 영역 기반 탐색) 방식이다. 이는 SLAM(Cartographer)으로부터 현재까지 생성된 2D 점유 격자 지도(Occupancy Grid Map)를 실시간으로 입력받는다.

알고리즘은 지도 데이터를 분석하여, 이미 인지된 공간(Known Space, 수치데이터:0)과 미지의 공간(Unknown Space, 수치데이터:-1)이 맞닿아 있는 경계 영역(Frontier)을 수학적으로 식별하고 추출한다.

추출된 다수의 경계 영역 중, 사전에 정의된 비용 함수(예: 현재 로봇 위치에서 가장 가까운 경계 지점)에 따라 최적의 탐색 지점을 동적으로 선정한다. 선정된 경계 지점의 좌표는 ROS2 내비게이션 스택(Nav2)의 새로운 목표 지점(Goal)으로 발행된다. Nav2는 이 목적지까지 장애물을 회피하며 로봇을 주행시키고, 로봇이 이동함에 따라 SLAM(Cartographer)은 맵을 갱신하며 미지의 영역을 확장해 나간다.

이 탐색-주행-맵핑 과정은 더 이상 탐색할 경계 영역이 지도 상에 존재하지 않을 때까지 반복적으로 수행되며, 이를 통해 덕트 환경 전체의 지도를 자율적으로 완성한다.

Ⅲ. 시스템의 제작[합성, 제작]

3.1 기본 제작
캡스톤디자인(1)을 수강하면서 프로젝트의 기반이 되는 하드웨어 플랫폼 선정 및 기본 소프트웨어 환경 구축을 중심으로 제작을 진행하였다.

Photo. 3-1. TurtleBot3 Burger

3.1.1 플랫폼 및 부품 선정
로봇 운영체제(ROS2)와의 호환성 및 오픈소스 생태계를 고려하여 [Photo. 3-1]의 터틀봇3 버거(TurtleBot3 Burger)를 기본 플랫폼으로 선정하였다. 주요 구성 요소로 메인 제어기인 Raspberry Pi 4, Dynamic Cell 모터와 모터 제어기인 OpenCR 보드, SLAM 구현을 위한 LDS-02 LiDAR 센서 , 그리고 결함 탐지를 위한 Raspberry Pi 카메라 모듈 등을 확정하였다.

3.1.2 개발 환경 구축
로봇 제어용 Raspberry Pi에는 Ubuntu Linux와 ROS2 Humble을 설치하였고, AI 영상 처리용 Raspberry Pi에는 Raspbian OS를 설치하여 역할을 분담하였다. 개발 PC 환경으로는 **WSL2(Windows Subsystem for Linux 2)**를 활용하여 ROS2 개발 환경을 구축하였다.

3.1.3 초기 S/W 구현 및 테스트
ROS2 환경에서 Cartographer SLAM, Nav2 내비게이션 스택, TurtleBot3 관련 패키지를 설치하고 기본적인 구동 테스트를 진행하였다. 또한, 케이블 결함 탐지를 위한 기초 데이터셋을 Roboflow 플랫폼을 통해 확보하고 초기 YOLO 모델 학습을 위한 환경을 구성하였다. 이 과정에서 WSL2 네트워크 설정 오류 , OpenCR 보드 펌웨어 인식 불가 등의 문제를 해결하며 기본 제작 단계를 완료하였다

3.1.2 케이블 손상 검출 시스템 개발

케이블의 손상 여부를 실시간으로 판별하기 위한 지능형 시각 인식 시스템 개발을 진행하였다. 본 시스템의 개발은 임베디드 엣지 디바이스의 연산 제약조건 하에서 실시간성(Real-time)을 확보하는 데 중점을 두었다.

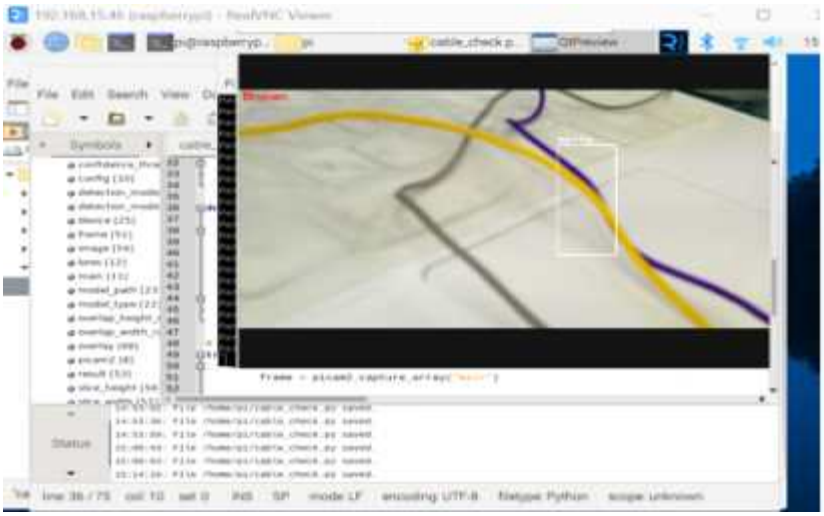

Photo. 3-2. SAHI 라이브러리 적용 후 케이블 손상체크 실행 시의 화면

Fig. 3-1. YOLO 기반 케이블 손상 객체 탐지 알고리즘

초기에는 SAHI 라이브러리를 적용하였으나, 추론 속도 저하 및 프레임 드랍 현상이 심각하여 실시간 적용에 부적합하다고 판단하였다.

따라서, 객체 탐지 알고리즘의 근본적인 변경이 필요하였다. 기존 R-CNN 계열처럼 영역 후보 기반의 두 단계(two-stage) 탐지 방식을 배제하고, 전체 입력 이미지를 단일 신경망으로 처리하여 객체들의 경계 상자와 클래스 확률을 한 번에 출력하는 단일 단계(one-stage) 딥러닝 모델인 YOLO(You Only Look Once) 객체 탐지 알고리즘을 채택하였다. 이 단일 단계 아키텍처는 복잡한 연산 파이프라인을 제거하여 월등히 빠른 추론 속도를 보장한다.

또한, 최종적으로 Raspberry PI 환경에서의 엣지 디바이스 연산 최적화를 통해 신속한 객체 탐지를 달성하고자, YOLO 모델 중 가장 경량화되고 추론 효율이 높은 YOLOv11 Nano 인공지능 모델을 주력으로 사용하여 모델을 개발하였다. 이 모델은 적은 연산 자원으로도 충분한 실시간 탐지 성능을 제공하기에 채택하였다.

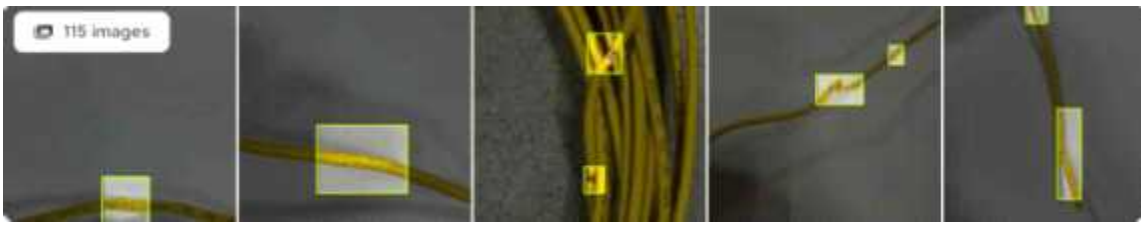

Photo. 3-3. 초기 케이블 결함탐지 데이터셋1

Photo. 3-4. 초기 케이블 결함탐지 데이터셋2

AI 기반 케이블 결함 탐지 기능의 기초 구현을 위해, RoboFlow 플랫폼 내에서 공개된 데이터셋 및 웹 크롤링을 통해 케이블의 피복 손상, 단선 등과 관련된 이미지 데이터를 총 1,500여 장 규모로 초기 확보하였다 ([Photo. 3-3], [Photo. 3-4] 참조). 확보된 초기 데이터셋에 대해 Roboflow 플랫폼에서 제공하는 기본적인 전처리 및 라벨링 작업을 활용하여 데이터셋 전처리작업을 수행하였다. 이후, Google Colaboratory 환경을 활용하여 확보된 데이터셋 기반으로 초기 YOLO(You Only Look Once) 모델 학습을 진행하였다.

3.1.3.2 SLAM 개발

Photo. 3-5. Rviz2를 통한 센서 데이터 시각화

[Photo. 3-5]는 LDS-02 LiDAR 센서로부터 실시간으로 수신되는 원시 포인트 클라우드 데이터를 ROS2 시각화 도구인 Rviz2 상에 렌더링한 결과이다. 이는 로봇 주행 중 센서 데이터의 정상 수신 여부와 주변 환경 인식 상태를 직관적으로 모니터링하기 위해 구현하였다.

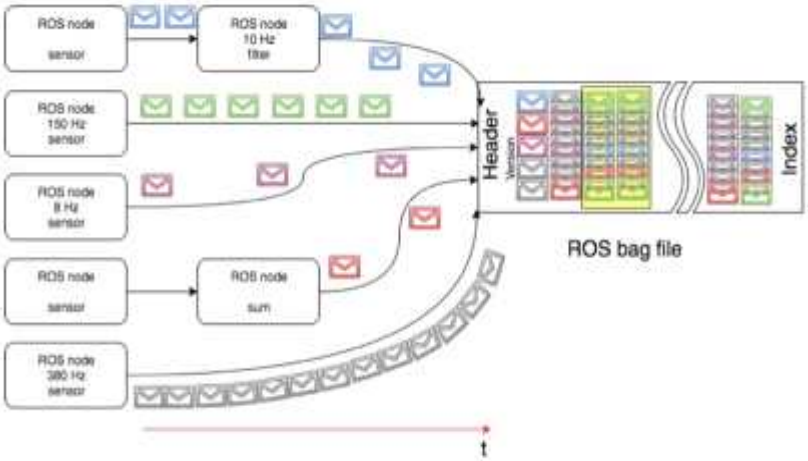

Fig. 3-2. 센서 데이터 녹화 과정

[Fig. 3-2]은 로봇의 실제 주행 중 LDS-02 LiDAR 및 IMU 센서 데이터를 ROS bag 파일(.bag) 형식으로 기록하는 과정을 나타낸다. 이렇게 저장된 센서 데이터 로그는 로봇 하드웨어 없이도 오프라인 환경에서 반복적으로 SLAM 알고리즘(예: Cartographer)의 파라미터 튜닝 및 맵핑 성능 검증을 가능하게 한다.

Fig. 3-3. Cartographer SLAM을 이용한 창학관 회로설계연구실 내부 맵핑 결과

[Fig. 3-3]은 창학관 회로설계연구실 내부 환경을 대상으로, LDS-02 LiDAR 센서로부터 획득한 데이터에 Cartographer SLAM 알고리즘을 적용하여 정상적인 2D 점유 격자 지도(Occupancy Grid Map) 작성을 성공적으로 완료하였다.

3.2 보충 제작 및 FeedBack 제작

2학기 및 하계방학 기간에는 1학기 기본 제작 결과와 교수님 피드백, 실제 테스트 결과를 바탕으로 반복적인 개선(Iterative Refinement) 중심의 보충 및 피드백 제작을 수행하였다.

3.2.1 하드웨어 피드백 기반 개선
3.2.1.1 터틀봇 개조
로봇 조립을 마친 후, 실제 사다리형 덕트 환경 테스트 결과, TurtleBot3 Burger의 표준 휠(Wheel)로는 주행 안정성 확보가 불가능하고 하단 센서 간섭 문제가 발생함을 확인하였다.

Photo. 3-6. TurtleBot3 Burger

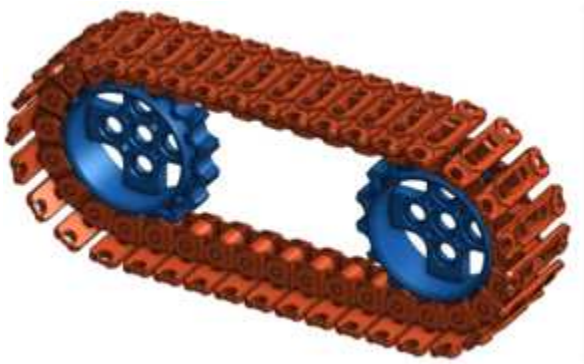

Photo. 3-7. TB3 캐터필러 트랙

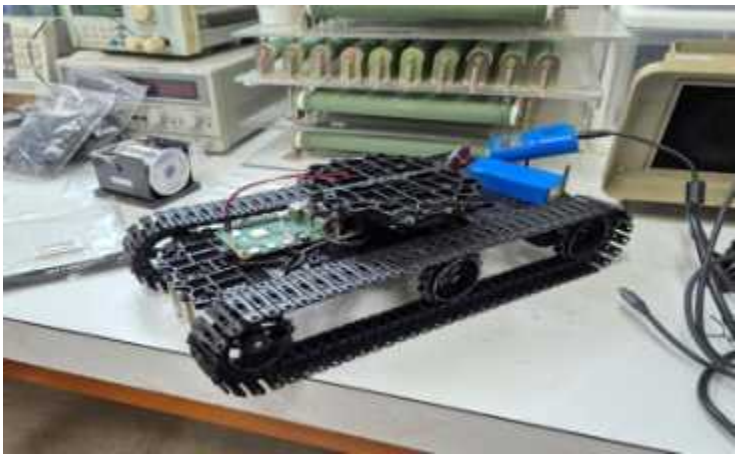

Photo. 3-8. 캐터필러 구동부 적용을 위한 1차 플랫폼 개조 형상

해당 피드백을 반영하여, 험지 주행 성능 강화를 위해 터틀봇 로봇의 구동부를 캐터필러 트랙방식으로 전면 개조하기로 결정하였고, 관련 부품(TB3 캐터필러 세트)[Photo. 3-7]을 구매하여 본래 3층 높이의 터틀봇 로봇을 1층으로만 구성하여 로봇 본체의 높이를 낮추는(Low-profile) 형태로 1차 개조를 진행하였다.

Photo. 3-9. 지름 4.5cm인 바퀴(좌) / 새로 설계한 지름 9cm 바퀴(우)

Photo. 3-10. 기존 로봇 바퀴와 새롭게 설계한 바퀴 비교

1차 캐터필러 개조 로봇의 덕트 환경 주행 테스트(시험) 결과, 낮은 지상고로 인한 LiDAR 센서의 물리적 간섭 및 주행 불안정성 문제가 피드백되었다.

이에 대한 1차 해결 방안으로, 기존 터틀봇 바퀴(지름 4.5cm) 대비 직경을 2배(9cm)로 확대한 바퀴를 자체 설계 및 3D 프린터로 제작하여 지상고를 확보하고자 하였다. ([Photo. 3-10]은 1차 대형 바퀴(좌)와 기존 바퀴(우)의 크기를 비교한다.)

그러나 이 1차 대형 바퀴를 장착하여 주행 테스트를 재수행한 결과, 바퀴 톱니(Sprocket)의 형상이 캐터필러 트랙의 내부 곡률과 불일치하는 기구적 결함이 새롭게 발견되었다.

이 결함으로 인해 주행 시 구동 토크가 트랙에 정상적으로 전달되지 못하고 주행 중 트랙 이탈이 발생하였으며, 결과적으로 좌회전, 우회전과 같은 조향이 불가능해지는 치명적인 2차 설계 오류를 확인하였다.

Photo. 3-11. 캐터필러 트랙 맞물림 개선을 위한 바퀴 톱니 (Sprocket) 형상 비교 (상: 1차 설계, 하: 2차 수정 모델)

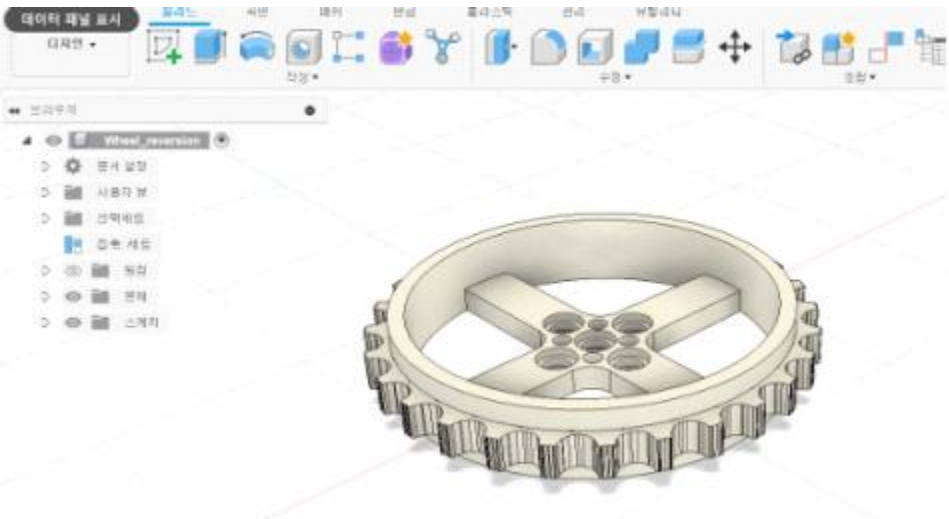

Photo. 3-12. 2차 수정한 로봇 바퀴

상기 피드백을 기반으로, 1차 설계의 기구적 결함을 해결하기 위해 Fusion 360을 이용한 구동 바퀴의 2차 재설계를 수행하였다 ([Photo. 3-12] 참조). 핵심 수정 사항은 바퀴 톱니(Sprocket)의 형상으로, 1차 설계 모델이 가지는 넓은 곡률 대신, [Photo. 3-13]의 2차 설계 바퀴 모델과 같이 캐터필러 트랙의 내부 링크와 정밀하게 맞물리도록(Meshing) 곡률을 재설계하였다.

Photo. 3-13. 2차 설계 바퀴와 1차 설계 바퀴의 캐터필러 맞물림 비교

재설계된 모델의 시제품을 3D 프린터로 출력하여 실증 테스트를 진행한 결과, 트랙과의 기계적 맞물림 성능이 현저히 개선되어 트랙 이탈 현상이 해소됨을 검증하였다. 검증이 완료된 2차 재설계 부품으로 로봇 구동부 전체의 바퀴를 교체 장착하였다.

추가적으로, 반복 주행 시 발생할 수 있는 바퀴 축의 마모 및 유격을 최소화하고 내구성을 강화하기 위해, 각 바퀴 축에 볼 베어링을 적용하고 로보티즈(ROBOTIS)의 2단 브라켓을 이용하여 프레임에 견고하게 고정하였다.

Photo. 3-14. 최종 개조 완료한 로봇 실물

최종적으로 2차 설계 바퀴를 적용하여 하드웨어 개조를 완수한 모습은 위 사진과 같다. 본 개조에서는 프레임의 강성 및 연결부의 구조적 안정성을 향상시키기 위하여 각 횡봉간 간격을 정밀히 조정하였으며, 로봇 본체의 캐터필러 트랙과 바퀴 사이의 동역학적 맞물림을 최적화하였다.

이에 따라 주행 시 구조물의 흔들림이 현저히 감소하였으며, 트랙의 미끄러짐이나 벗어남 현상이 실험적으로 발생율이 줄어들었음을 확인하였다. 더불어, 이러한 설계 개선은 로봇의 실질적인 덕트 내 이동 신뢰성과 반복 작업에서의 내구성을 증대시키는 데 중요한 기여를 하였다. 앞으로도 동일한 원리를 다양한 환경 조건 하에서 반복적으로 적용함으로써, 현장 실증 단계에서의 하드웨어 신뢰성을 체계적으로 확보하고자 하였다.

3.2.1.2 덕트(DUCT) 모의 환경 구축

Photo. 3-15. 초기 확보한 일자형 덕트(DUCT)

1학기 기본 제작 단계에서는 실제 덕트 환경에서 로봇의 주행성과 기초 조작 성능을 검증하고자 하였으며, 이를 위해 [Photo. 3-15]와 같이 KT 통신실의 일자형 사다리형 트레이를 대여하여 해당 덕트를 대표적인 시험 환경으로 선정하였다.
해당 트레이는 실제 현장에 설치된 사다리형 덕트와 동일한 구조, 재질 및 간격을 가지고 있어 로봇의 하드웨어 설계 변경 시 고려해야 할 요소 및 실질적으로 발생할 수 있는 구조적·운영적 문제점을 다각적으로 분석하는 데 적합하였다. 본 시험 환경은 약 1m 구간의 직선형 주행로를 제공하였으며, 이를 통해 로봇의 전진·후진(Forward/Backward) 동작, 시작 및 정지 제어, 회전 및 방향전환 성능을 반복적으로 계측하여 초기 설계의 신뢰성을 실증적으로 검토하였다.

이를 바탕으로 구조적 간섭, 미끄러짐, 주행 중 정렬 오차 등 덕트 주행에서 발생 가능한 문제점들을 파악하였고, 향후 하드웨어·소프트웨어 최적화 방향을 설정하는 데 중요한 토대를 마련하였다.

개조한 하드웨어가 실제 덕트 트레이 위에서 주행하는 과정에서 캐터필러 트랙의 미끄러짐 현상이 발생하여 로봇이 전후진 동작을 제대로 수행하지 못하는 문제를 발견하였다. 이러한 미끄러짐은 트랙과 덕트 표면 간 마찰력 부족으로 인해 발생하는 것으로 분석되었으며, 이는 곡면 혹은 평탄하지 않은 환경에서 자율주행 로봇이 흔히 직면하는 대표적 기술적 과제 중 하나임이 확인되었다.

이를 해결하고자 캐터필러 트랙과 직접 결합할 수 있는 고무패드를 선정 및 확보하여 추가로 장착하였다. 결합 이후 다시 주행 테스트를 반복적으로 진행한 결과, 이전보다 현저하게 원활한 주행성과 정밀한 제어 응답을 확보할 수 있었다. 이에 따라 전후진은 물론 정지·회전 동작 또한 더 신속하고 안정적으로 수행되는 것이 실험적으로 확인하였다.

그러나 확보한 일자형 덕트 트레이의 경우 구조적으로 직선 구간만으로 구성되어 있어, 좌/우회전과 같은 로봇의 조향 성능 및 SLAM 알고리즘의 코너링 맵핑 기능을 테스트하고 검증할 수 없는 명확한 한계를 발견하였다. 이 피드백은 2학기 모의 덕트 환경을 구축하는 의사결정에 직접적인 영향을 주었다.

Photo. 3-16. 2차 복합 테스트를 위해 구축된 H자형 모의 덕트 환경

1차 직선형 덕트 환경(Photo. 3-14) 테스트 환경의 한계를 극복하고 실제 산업 현장의 복잡성을 모사하기 위하여, [Photo. 3-16]와 같이 사다리형 케이블 수평티 트레이 2개를 구매 및 조립하였다. 이를 통해 H자형의 복합 모의 덕트 테스트 환경을 완성하였으며, 실제 환경과 유사하게 내부에 케이블 다발을 설치하였다. 2차 모의 덕트 환경은 개조된 로봇의 하드웨어 주행성, SLAM 알고리즘, 자율주행 기능 등 모든 핵심 모듈의 성능 검증을 위한 테스트 환경으로 활용하였다.

3.2.2 SLAM 및 자율주행 시스템 고도화

3.2.2.1 SLAM 알고리즘 개선

1학기부터 사용한 Cartographer SLAM의 높은 자원 소모 문제를 해결하고자, 2학기 초 'SLAM_Toolbox'로의 전환을 시도하였다. 그러나 'SLAM_Toolbox'가 센서 데이터는 수신하나 지도로 변환하지 못하는 문제가 발생하여, 수차례의 시도 끝에 다시 안정성이 검증된 Cartographer SLAM으로 복귀하는 결정을 내렸다.

Photo. 3-17. SLAM_ToolBox를 이용한 mapping 사진

그러나 'SLAM_Toolbox' 적용 테스트 결과, LiDAR 센서 데이터는 정상적으로 수신하였으나 이를 지도로 변환하는 과정에서 오류가 발생하여 맵 생성이 이루어지지 않는 문제가 지속적으로 관찰되었다. 수차례의 파라미터 튜닝 및 디버깅 시도에도 불구하고 문제 해결이 지연되었고 개발 일정 준수 및 시스템 안정성 확보를 위해 최종적으로 기존의 Cartographer SLAM 알고리즘으로 복귀하는 결정을 내렸다.

explorer.py

▼ source

```
class ExplorerNode(Node):
    def __init__(self):
        super().__init__('explorer')
        self.get_logger().info("Explorer Node Started")

        # --- \uc0c1\ud0dc \uad00\ub9ac \ubcc0\uc218 ---
        # 'EXPLORING': \ud0d0\uc0c9 \uc911
        # 'RETURNING_HOME': \uc2dc\uc791 \uc9c0\uc810\uc73c\ub85c \ubcf5\uadc0 \uc911
        # 'SHUTTING_DOWN': \uc885\ub8cc \uc911
        self.state = 'EXPLORING'
        self.get_logger().info(f"Initial Setting. state : EXPLORING")
        self.start_position = None # \ud0d0\uc0c9 \uc2dc\uc791 \uc704\uce58\ub97c \uc
        self.frontier_failure_count = 0

        # Subscriber to the map topic
        self.map_sub = self.create_subscription(
            OccupancyGrid, '/map', self.map_callback, 10)
```

Fig. 3-4. Explorer(자율탐색 알고리즘) 코드 일부

Cartographer를 통해 덕트 모의 환경의 2D 점유 격자 지도 작성을 완료한 후, Nav2 내비게이션 스택과 프론티어 기반 탐색 알고리즘(Frontier-Based Exploration)인 Explorer 패키지를 통합하여 자율주행 기능 구현을 시도하였다.

그러나 실제 주행 테스트 과정에서, 로봇이 지도상의 장애물을 인지하고도 이를 회피하지 못하고 지속적으로 충돌하는, 주행 안정성이 결여되는 심각한 문제가 관측되었다.
이에 대한 원인 분석 결과, 이는 개조된 로봇의 비대칭적 윤곽(footprint) 설정값이 실제 기구부와 일치하지 않았으며, 동시에 협소한 덕트 환경의 특성이 Nav2의 Costmap 및 경로 추종 관련 파라미터에 최적화되지 않았기 때문으로 규명되었다.

```
footprint: "[[0.285, 0.12], [0.285, -0.12], [-0.285, -0.12], [-0.285, 0.12]]"
plugins: ["obstacle_layer", "voxel_layer", "inflation_layer"]
alpha1: 0.3
alpha2: 0.3
alpha3: 0.3
alpha4: 0.3
alpha5: 0.2
max_particles: 2500
min_particles: 500
```

Fig. 3-5. 수정한 파라미터(1)

```
xy_goal_tolerance: 0.10
sigma_hit: 0.15
recovery_alpha_fast: 0.01
recovery_alpha_slow: 0.01
inflation_layer:
  plugin: "nav2_costmap_2d::InflationLayer"
  inflation_radius: 0.05
  ObstacleFootprint.scale: 0.4
  PathAlign.scale: 38.0
```

Fig. 3-6. 수정한 파라미터(2)

상기 관측된 주행 충돌 문제를 해결하기 위해, Nav2 스택의 핵심 설정값에 대한 체계적인 튜닝을 수행하였다.
먼저 로봇의 기구적 윤곽을 정밀하게 반영하도록 footprint 파라미터를 캐터필러 기반의 대칭형 모델로 재설정하였으며 협소한 덕트 환경에서의 안정성을 확보하기 위해 로컬 플래너의 주요 파라미터를 수정하였다.

구체적으로 목표 도달 허용 오차, 장애물 회피 가중치와 경로 추종 가중치 관련 파라미터(xy_goal_tolerance, ObstacleFootprint.scale, PathAlign.scale)의 값을 반복적인 주행 실험 및 테스트 과정을 거쳐 최적화를 진행하였다. 이러한 파라미터 최적화 과정을 통해, 주행 중 장애물과 충돌하지 않는 안정적인 자율주행 기능을 구현하였다.

3.2.3 소프트웨어 아키텍처 개선

3.2.3.1 AI 모델 고도화

1학기 AI 기반 케이블 결함 탐지 모델 운용 과정에서, 단선 케이블 상태를 일반 손상 범주와 구분하지 못하는 한계점이 발견되었다. 이는 모델의 학습 데이터셋이 단선 이미지를 충분히 포함하지 못하여, 실제 배선 환경에서 중요한 단선 결함을 효과적으로 식별할 수 없었기 때문임을 분석하였다.

이에 단선 케이블의 다양한 위치・각도・배경 조건에서 촬영된 원천 이미지와 어노테이션 데이터를 체계적으로 추가 구축하였으며, 이를 기반으로 AI 모델의 재학습을 수행함으로써 분류 성능을 실질적으로 향상시켰다.

Photo. 3-18. 추가적으로 학습시킨 데이터셋 및 라벨링

Time	Total Frames	FPS	Latency (ms)	Bandwidth (kbps)
1분	599.98	10.0015	2109.13	225.109
3분	1799.93	10.0007	1511.41	225.076
6분	3599.78	10.0001	910.89	225.034
9분	5399.95	10.0002	310.87	225.041
10분	5999.84	10.0001	110.87	225.029

Fig. 3-7. M4 Processor에서의 RTSP 실험결과

또한, 기존 Raspberry Pi 환경에서 직접적인 딥러닝 추론 연산을 수행하던 방식은 하드웨어 자원 한계로 인한 실시간성 저하 및 프레임 지연 현상이 빈번하게 발생하였다. 이러한 병목을 해소하기 위하여, 모델 추론 연산을 AWS EC2 클라우드 서버 또는 M4 프로세서가 장착된 고성능 노트북으로 분산하여 처리하는 시스템 아키텍처로 재구성하였다. 이 방식은 센서로부터 취득한 이미지를 네트워크를 통해 서버 또는 노트북으로 전송한 후, 추론 결과를 다시 로봇 제어기로 전달하는 구조를 기반으로 한다. 위의 도식화를 통해 M4 processor에서의 RTSP 프로토콜 지연시간을 측정해본 결과 시간이 지날수록 1초 미만의 지연시간이 측정되어 실시간 동작 안정성이 현저히 향상됨을 검증하였다.

3.2.3.2 웹 기반 실시간 모니터링 시스템

Photo. 3-19. 제작한 로봇 제어 웹 UI/UX 화면

위의 [Photo. 3-19]는 Flask 서버를 기반으로 개발한 로봇 통합 제어 시스템의 웹 UI/UX 초기 화면을 나타낸다. 본 웹 시스템은 로봇의 핵심 상태 정보를 종합적으로 모니터링이 가능하며, 로봇의 배터리 충전 현황, ROS 통신 연결 신뢰도, 실시간 Odometry 데이터, 센서 오류 감지 등의 주요 운영 지표를 직관적으로 시각화하였다. 사용자는 수동조작 모드와 케이블의 손상 및 단선 여부 실시간 판별, 그리고 로봇의 주행 데이터를 기반으로 동적으로 구성되는 SLAM 지도의 확인 기능을 선택적으로 활용할 수 있도록 메뉴 구조를 설계하였다.

```
app = Flask(__name__)
capture = cv2.VideoCapture(-1)
capture.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
capture.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)
```

Fig. 3-8. 카메라 설정 및 영상 스트리밍 코드

```
def gen_frames():
    while True:
        success, frame = capture.read()
        if not success:
            break
        else:
            ret, buffer = cv2.imencode('.jpg', frame)
            frame = buffer.tobytes()
            yield (b'--frame\r\n'
                   b'Content-Type: image/jpeg\r\n\r\n' + frame + b'\r\n'
```

Fig. 3-9. 영상프레임 생성 함수 코드

```
@app.route('/video_feed')
def video_feed():
    return Response(gen_frames(), mimetype='multipart/x-mixed-replace; boundary=fr
```

Fig. 3-10. gen_frames() 함수에서 생성된 프레임을 반환 코드

본 프로젝트에서는 웹 시스템의 경량화, 유연한 구조 설계, 신속한 프로토타이핑, 그리고 다양한 외부 파이썬 라이브러리와의 호환성을 극대화하고자 Flask 프레임워크를 채택하였다. Flask는 모듈화된 마이크로 프레임워크 구조로 인해 기능 확장과 유지보수가 용이하며, 실시간 로봇 데이터 연동 및 사용자 맞춤형 UI 구현에 최적화된 개발 환경을 제공하였다.

홈 메뉴에서는 로봇의 CV 모듈 및 SLAM 모듈 연결 상태, 배터리 잔량, 실시간 위치 및 방향정보 등을 직관적으로 확인할 수 있도록 설계하였다.
특히, 실시간 영상 스트리밍 기능 구현을 위해 위의 [Fig. 3-8] 및 [Fig. 3-9] 이미지는 Flask 기반 웹 서버와 OpenCV를 연동하여 JPEG 인코딩된 카메라 프레임을 HTTP multipart 방식으로 지속적으로 전달하도록 하였다.

[Fig. 3-10] 이미지는 Flask의 라우팅 기능과 Response 객체를 활용해 멀티파트 미디어 타입으로 실시간 비디오 스트림을 웹에 제공하는 핵심 코드 구조를 보여준다. 이러한 설계는 최소한의 지연으로 고신뢰성 영상 처리와 실시간성 모니터링을 동시에 추구하도록 시스템을 최적화하였다.

Photo. 3-20. 웹 내의 수동조작 메뉴 화면

수동조작 메뉴는 웹 기반 인터페이스를 통해 사용자가 원격 위치에서 로봇을 실시간으로 직접 조작할 수 있도록 설계되었다. 클라이언트(브라우저)에서 입력된 명령 신호는 Flask 서버의 /control 엔드포인트를 통해 Web Application으로 전달되며, 이 과정에서 각 주행 방향에 대응하는 직관적인 방향키 UI와 실시간 영상 피드백 기능이 통합되어 있다. 이러한 명령은 내부적으로 ROS 스레드, 이미지 스레드, DB 스레드 등으로 전달되고, ROS 클라이언트와 즉시 연동되어 하드웨어 동작으로 빠르게 반영된다.

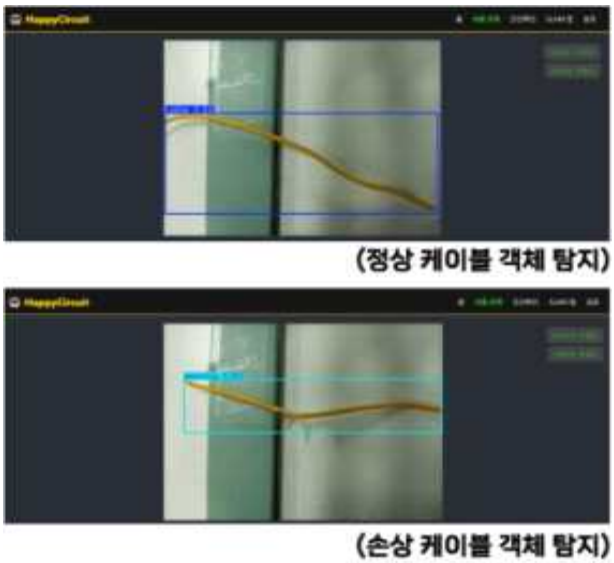

Photo. 3-21. 웹 상 케이블 객체 탐지 확인 기능

단선 확인 메뉴는 Raspberry Pi에 연결된 카메라 모듈을 이용하여 실시간으로 취득된 영상 프레임 내에서 케이블의 손상 유무를 자동으로 판별하는 인공지능 기반 딥러닝 추론 시스템을 활용하여 케이블의 손상 유무를 확인할 수 있도록 구현하였다.

본 시스템은 영상 프레임 속 케이블 객체를 탐지한 후, 정상과 손상(단선) 상태를 구분하여 웹 UI 상에 직관적으로 시각화하며, 각 결과는 객체 바운딩 박스를 통하여 실시간으로 표기된다.

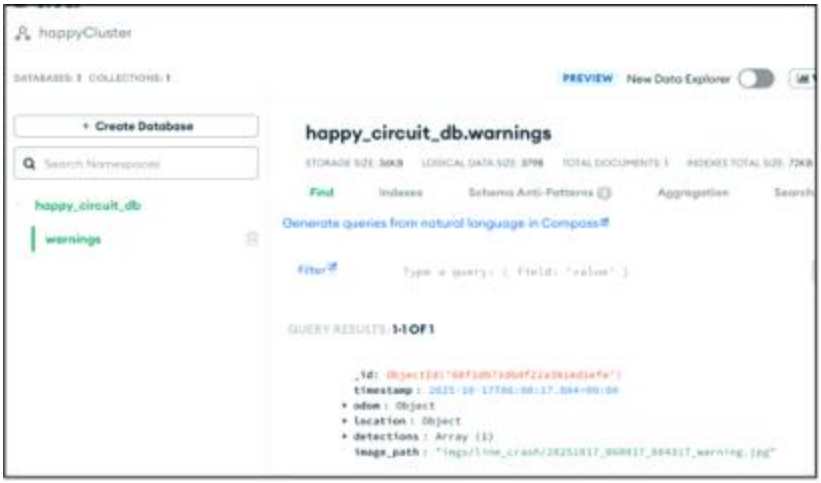

Photo. 3-22. Atlas DB에 저장된 손상 케이블 데이터

주행 중 케이블 손상 또는 단선이 탐지될 경우, 로봇 제어 시스템은 해당 이벤트의 발생 시점에 로봇의 위치 정보, 감지 시간, 그리고 현장 상황을 촬영한 이미지를 실시간으로 MongoDB Atlas 클라우드 데이터베이스에 자동 저장하도록 구현하였다.

각 데이터는 구조화된 JSON 포맷으로 기록되며, 데이터베이스 내 warnings 컬렉션에는 타임스탬프, 주행 위치(odom), 감지 객체 정보, 이미지 파일 경로 등 주요 상태 변수를 함께 포함한다. 이를 통해 손상된 케이블 데이터를 저장 및 관리가 가능하다.

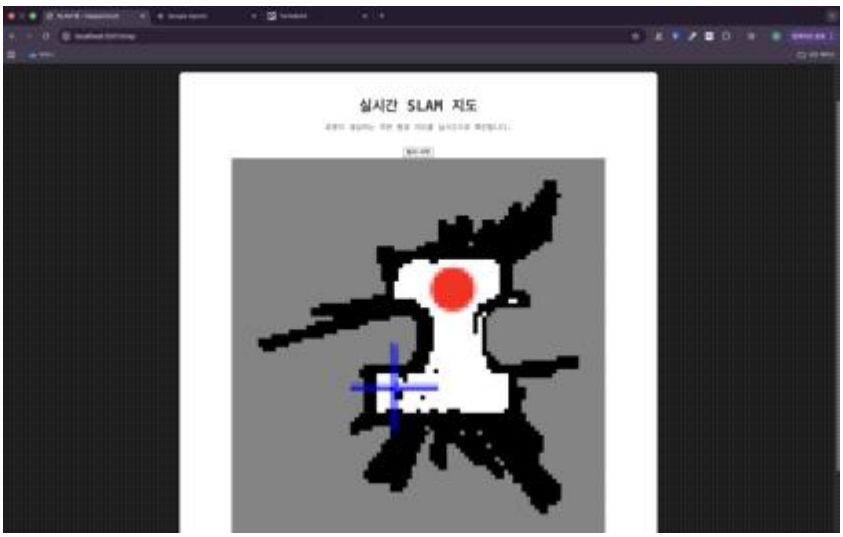

Photo. 3-23. 웹상에서 덕트 내부 mapping 지도 확인

SLAM(Simultaneous Localization and Mapping) 알고리즘을 통해 실시간으로 생성되는 지도(Map) 데이터를 JSON 포맷으로 웹 클라이언트에 전달하고 시각화하는 시스템을 구현하였다. 본 시스템은 덕트 내부 환경을 주행하며 취득한 LIDAR 및 Odometry 데이터를 기반으로 실시간 지도 작성과정을 웹페이지 상에 동적으로 표출하며, 사용자는 별도의 추가 프로그램 설치 없이 브라우저를 통해 지도 생성 결과를 손쉽게 확인할 수 있다. 또한, 지도상에는 현재 로봇의 위치를 실시간으로 반영하여 붉은 원으로 표시하고, 로봇의 출발 위치는 청색 십자가로 구별하여 현장 운영자의 공간 인지 및 실시간 작업 관제가 용이하도록 설계하였다.

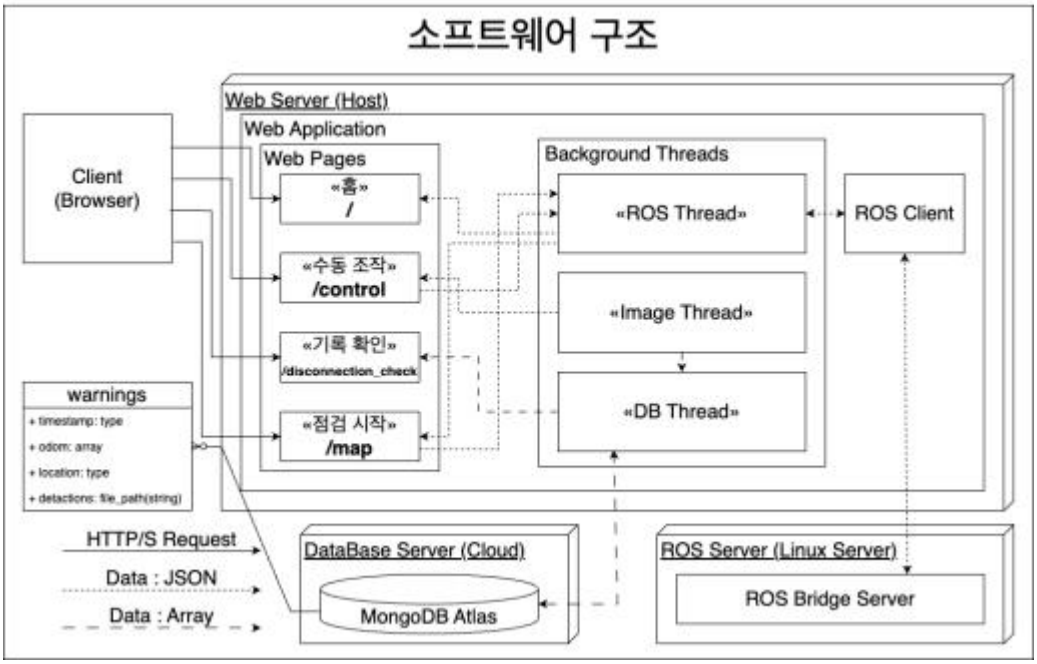

Fig. 3-11. 전체적인 소프트웨어 구조

위의 [Fig. 3-11]의 소프트웨어 구조도는 AI 기반 스마트 덕트 관리 로봇 시스템의 통합적 웹 플랫폼 구성과 데이터 처리 흐름을 체계적으로 도식화한 것이다.
각 웹 페이지에서의 사용자 명령 또는 모니터링 신호는 내부 백그라운드 스레드(ROS, 이미지, DB)로 전달되어, ROS 클라이언트를 통한 로봇 주행 명령 및 센서 데이터 수집, 이미지 인식/추론, 데이터베이스 연동 등 각종 작업이 비동기적으로 처리된다.

ROS 서버는 Linux 기반에서 동작하는 ROS Bridge 서버를 통해 실제 물리 로봇 및 SLAM, 센서 데이터와 연계된다. 데이터베이스 레이어는 MongoDB Atlas와 같은 클라우드 DB를 채택하여 경고(warnings) 발생 시점의 타임스탬프, 위치(odom), 감지 객체, 이미지 경로 등 주요 정보를 구조화된 JSON 데이터로 저장한다. 이러한 구조는 RESTful API 및 웹소켓 기반의 실시간 통신, 모듈 간 데이터 독립성, 장애 이력 관리 및 통계적 분석의 용이성을 동시에 달성하도록 최적화되었다.

Ⅳ. 시스템의 동작 검증[시험, 성능평가]

본 단계에서는 하드웨어 및 소프트웨어 단위 모듈별 정상 동작을 체계적으로 검증하고, 전체 시스템 성능에 대한 정량 및 정성적 평가를 실시하였다.

4.1 하드웨어 동작 검증

Photo. 4-1. 2차 설계 바퀴와 1차 설계 바퀴의 캐터필러 맞물림 비교

2차 설계에서 적용한 바퀴 및 캐터필러 트랙 시스템의 구조적 결합 및 동작 신뢰성을 정량적·정성적으로 검증하였다. 설계 변경 후의 바퀴는 실제 캐터필러 트랙과의 정밀한 맞물림 구조를 갖추도록 3D 프린팅 및 소재 보강, 지름·폭 최적화 등을 거쳐 제작되었다. 바퀴 표면의 지지 체결부와 캐터필러 삽입홈의 위치 정밀도를 강화하고, 연결부의 반복 내구시험을 실시하였다. 그 결과, 주행 및 회전 시 이탈·슬립 없이 표준 단면을 안정적으로 유지함을 확인하였다.

Photo. 4-2. 바퀴 연결부의 반복 내구시험

실제 덕트 환경 및 평면 주행 시험을 반복적으로 실시한 결과, 2차 개량된 바퀴를 탑재한 로봇은 동적 하중 조건에서도 트랙 이탈이나 미끄러짐 없이 기계적 결합이 견고하게 유지되는 것을 확인하였다. 주행 도중 바퀴와 캐터필러 사이의 장력, 삽입 홈 정렬, 반복 구동 내구성 등에 대한 점검을 병행하였으며, 주행 중 일관된 구동력과 직진성을 확인함으로써, 구조적 고신뢰성을 실증하였다.

또한, 동등 환경에서 SLAM(Simultaneous Localization and Mapping) 기반 자율주행 테스트를 수행한 결과, 연구실 내부 공간의 약 50% 영역에 대해 로봇의 위치추정, 장애물 감지, 지도작성 등 각 항목에서 정상적으로 매핑 결과가 도출되었음을 확인하였다.

바퀴 변형 과정에서는 추가적으로 베어링과 와셔를 조합하여 축 방향 및 회전 방향 안정성을 확보하였다. 베어링은 바퀴축의 회전 마찰을 최소화하면서 하중과 진동을 견고하게 견디도록 배치하였고, 와셔는 베어링 내륜과 샤프트 숄더 사이 또는 너트와 함께 설치하여 축방향 위치 이동과 풀림 현상을 효과적으로 억제하고자 이중너트 체결을 통해 개조하였다.

4.2 소프트웨어 동작 검증

4.2.1 Raspberry PI Camera 웹소켓 연결

케이블 단선체크 전용 라즈베리파이에 개별 포트를 구성하고, Flask 웹서버와 라즈베리파이 카메라 간 웹소켓 소통 체계를 설계하여 실시간 비디오 스트리밍을 성공적으로 구현하였다. 이 과정에서 Flask 프레임워크 특유의 비동기 처리 구조를 적용하여 영상 프레임의 수신·렌더링 지연을 최소화했으며, 실험적으로 안정성과 프레임율, 데이터 응답성을 반복 검증하였다.

Photo. 4-3. 웹소켓 연결된 모습

4.2.2 덕트 환경 내에서의 주행과 SLAM Mapping 테스트

실제 덕트 환경 내에서 주행 안정성과 SLAM 기반 실시간 지도 작성 기능의 성능을 집중적으로 평가하였다. 설계된 주행 알고리즘은 직진, 회전, 장애물 회피 등 다양한 운행 동작이 덕트 내에서 안정적으로 구현되는지 반복적으로 테스트되었으며, 센서 데이터(odometry, LIDAR 등)와 ROS2 네비게이션 알고리즘의 실시간 연동을 통해 로봇 위치추정, 동선 변경, 자동경로 학습 등 기능의 정상동작 테스트를 진행하였다.

Photo. 4-4. 덕트 데모 테스트 환경

Photo. 4-5. Mapping 진행 중인 터틀봇

로봇 바퀴의 지름을 2배로 확장하기 전 구비한 실제 사다리형 트레이 환경에서 주행 테스트 결과, 주행 불안정 및 하단부 레이저 센서가 트레이에 부딪히는 문제 발생하였고 해당 문제점으로 인해 정상적인 맵핑이 불가능한 상황이 발생하였다. 또한 더욱 빠른 Mapping을 위해 도입한 SLAM_Toolbox의 경우 지도 생성(Mapping)이 되지 않는 문제 지속되었다.

해당 문제점을 개선하고자 개발 전략을 변경하여, 이전에 사용했던 'Cartographer SLAM'으로 복귀 결정하였고, 로봇의 지상고를 높여 센서의 물리적 간섭을 없애고 주행 안정성을 확보하기 위해 바퀴 크기 확장 결정하였다. 개선 이후 연구실 내부공간에 대해 SLAM을 통해 Mapping을 다시 진행했을 때 [Photo. 4-6]와 같이 정상적으로 Mapping되는 것을 확인하였다.

Photo. 4-6. 연구실 내부 mapping 결과

4.2.3 AI 기반 케이블 결함 탐지 알고리즘 검증 분석

Photo. 4-7. 실시간 케이블 손상체크 추론 사진
초기 학습된 YOLO 모델을 Raspberry Pi에 탑재하여 실시간 추론을 진행하며 1차 성능 검증을 수행하였다.

초기 검증 단계에서 두 가지 주요 한계점이 식별되었다. 첫째, 일부 손상 케이블은 탐지하였으나 전체적인 탐지 정확도(Accuracy)가 안정적이지 못하였다. 둘째, [Photo. 4-8]와 같이 명백하게 절단된 단선' 케이블을 '손상' 클래스로 분류하지 못하고 정상 케이블로 오인하는 치명적인 분류 오류가 발생하였다.

Photo. 4-8. 추가적으로 학습시킨 데이터셋 사진

이러한 모델의 분류 실패 문제를 해결하기 위해, 단선 시나리오를 포함한 고해상도의 케이블 데이터셋을 [Photo. 4-8]와 같이 추가로 촬영 및 제작하였다. 이후, 보강된 데이터셋을 바탕으로 모델 재학습을 진행하여 단선 케이스에 대한 탐지 강건성을 확보하고자 하였다.

또한, Raspberry Pi의 본질적인 연산 성능 한계로 인한 낮은 추론 속도(FPS) 문제를 해결하기 위해, 시스템 아키텍처의 수정을 결정하였다. AI 추론 연산의 주체를 기존의 엣지 디바이스(Raspberry Pi)에서 M4 프로세서가 탑재된 고성능 노트북을 활용하여, 실시간 추론 속도와 탐지 정확도를 동시에 확보하는 방향으로 시스템을 개선하였다.

4.2.4 Explorer 자율주행 알고리즘 검증 및 평가

Explorer 자율주행 알고리즘의 검증에서는 두 가지 ROS2 SLAM 패키지, Cartographer와 SLAM_toolbox의 Occupancy Grid Map 표현 방식 및 자율주행 실행 결과를 비교 분석하였다.

Cartographer는 확률 기반의 Occupancy grid map을 활용하여 각 격자에 다양한 확률값(0~100)을 할당하는 구조로, 미탐색 및 접근 가능한 구간을 연속적이며 세밀하게 표현할 수 있으나, 실제 프론티어 탐색(Frontier-based exploration) 시 특정 위치의 탐색 가능 여부를 명확히 이진 판단하기 어렵고, 남아있는 저확률 영역(예: Occupancy grid map : 30 등)이 실제 주행 가능한 영역이 아닌 경우에도 프론티어로 선택되는 문제가 발생하였다.

SLAM_toolbox는 Occupancy grid의 값을 3가지(-1: 미탐색, 0: 탐색 가능, 100: 장애물)로 단순 명확하게 이산 할당하여, 자율주행 로직에서 프론티어 선정 기준이 일관되고 직관적으로 적용된다. 실험 결과, Cartographer 적용 시 프론티어 선정에 소요되는 시간이 매우 길어지고, 탐색 대상 수가 급증(600개 이상/5~10초)하여 전체 자율주행의 진행 속도가 크게 저하되는 현상을 관찰하였다.

반면 SLAM_toolbox에서는 60개 내외의 프론티어만이 생성되어 자율주행 경로의 계획 및 실행에서 신속성을 확보할 수 있었으며, 실제 주행 결과도 더욱 완만하고 안정적으로 진행됨을 확인하였다.

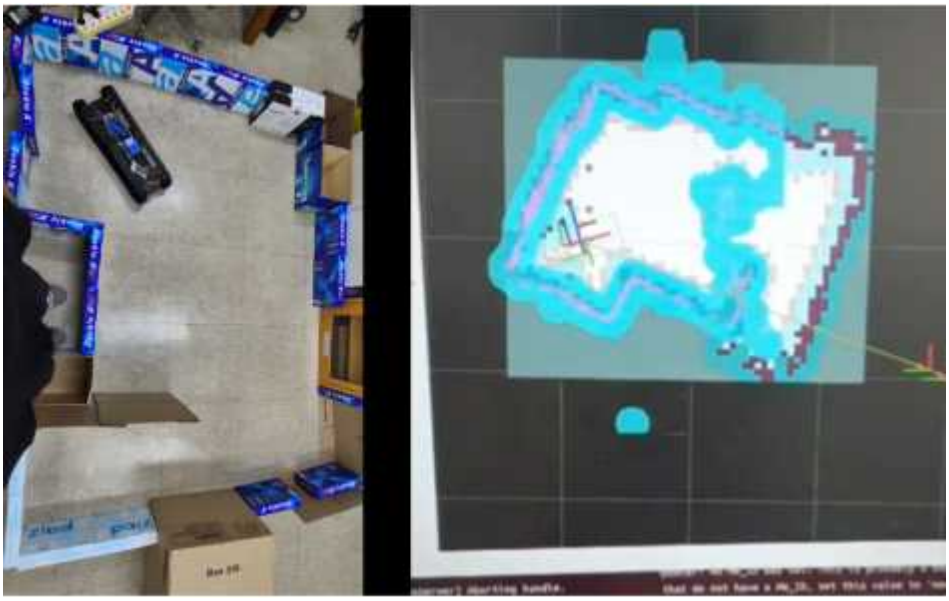

Photo. 4-9. SLAM_Toolbox & Explorer 자율탐색 알고리즘 기반 자율주행

이러한 분석을 바탕으로, 본 시스템의 덕트 환경 탐사에는 SLAM_toolbox 패키지가 자율주행 효율성과 안정성 측면에서 Cartographer에 비해 우수함을 실험적으로 확인하였으며, 최종적으로 SLAM_toolbox 기반 explorer 자율주행 알고리즘을 채택하여 구현하였다. 이로써 환경 탐색의 신뢰성과 운영 효율성 증대, 실시간 맵 업데이트 및 프론티어 기반 경로 생성에서의 성능적 이점을 확보할 수 있었다.

V. 결론 [결과도출]

졸업작품의 최종 시스템은 초기 목표로 설정한 덕트 환경 내 자율주행 및 케이블 결함 자동 검출 기능, 그리고 원격 웹 기반 통합 관제의 구현이라는 세부 요구사항을 모두 만족하는 방향으로 실증적 완성도를 입증하였다.

하드웨어 측면에서는 실제 덕트 주행을 고려한 캐터필러-바퀴 구조의 내구성, 베어링 축 고정 방식, 각종 센서 및 카메라 인터페이스가 반복적 시험에서 안정적으로 동작하는 것이 확인되었으며, 소프트웨어 검증에서는 ROS2와 SLAM_toolbox를 결합한 자율주행 알고리즘의 공간 맵핑 정확도, 프론티어 탐색 효율, 실시간 경로 재설정 등 모든 주요 기능이 산업적 요구 수준을 충족했다.

구분	기능	설명	현재진척도(%)
S/W	위치정보 추적	IoT 를 활용하여 위치 및 환경 데이터를 실시간 수집하고, 오픈소스 H/W 및 S/W 와 연동하여 로봇의 위치정보를 제공하는 기능.	100
	AI 데이터분석 및 케이블 손상체크	덕트 내부 환경을 실시간으로 분석하고, 이상 징후(과부하, 온도 상승, 케이블 손상체크 등)를 감지하는 AI 기반 데이터 분석 기능	100
	SLAM	Simultaneous Localization and Mapping 기술을 적용하여, 덕트 내부에서 로봇이 실시간으로 위치를 추적하고 지도를 작성하는 기능	100
	원격제어 및 진단	로봇의 실시간 제어 및 상태 진단이 가능하도록 클라우드 및 네트워크 기술 기반 원격 유지보수 기능 제공	100
	자율주행	SLAM 과 LiDAR 센서를 활용해 덕트 내부를 실시간으로 정밀하게 매핑하고, 로봇의 위치를 정확히 추정하며 자율적으로 주행 장애물 회피 및 최적 경로 탐색 기능을 통해 복잡하고 협소한 환경에서도 안정적인 이동과 점검을 수행	100
H/W	덕트 모델환경	상업 및 산업용 건물 내 덕트 구조를 모델링하고, 다양한 환경에서 설계한 덕트 관리 로봇을 테스트할 수 있도록 모의 환경을 구축	100
	이동체 / 로봇	지면 뿐 만 아니라 덕트의 내부 크기에 상관없이 원활히 이동할 수 있는 자율주행 기반 이동체(로봇) 개발	100

Table. 5-1. 목표 기능 대비 개발 완료 기능의 성능 및 진척도 종합 비교표

상기 비교표는 본 졸업프로젝트에서 초기에 설정된 시스템 핵심 기능과 실제 개발 완료된 결과물의 기능별 완성도, 진척율, 성취 여부를 객관적으로 제시한다. 각 항목은 S/W 및 H/W로 나뉘어 위치정보 추적, AI 데이터분석 및 케이블 결함 진단, SLAM 기반 지도 작성, 원격제어・진단, 자율주행, 덕트 모델 현장적합성, 이동체/로봇 완성도 등 총체적 시스템 관점에서 분류・서술된다.

표에 따르면 모든 주요 기능들이 목표 설정 당시 요구한 수준 및 산업 현장에서 요구되는 기술적 완성도 기준을 100% 충족한 것으로 평가된다. IoT・센싱과 AI 진단, 실시간 맵핑 및 자율주행 알고리즘, 웹 연동・원격 관제, 덕트 내구 테스트, 이동 메커니즘 등에서 실제 실험・현장 검증을 통해 정량적/정성적 요구지표 모두가 달성되었음을 명확히 보여준다.

5.1 기대효과

(1) 기존 서비스와의 차별성

- 실시간 자동화 점검
  SLAM과 LiDAR 센서를 활용하여 Explorer 자율탐색 알고리즘을 기반으로 자율주행 로봇을 통해 덕트 내부를 정밀하게 실시간 매핑하고, 장애물 회피 및 이상 징후 감지까지 자동으로 수행합니다. 기존의 수동 점검 방식 대비 인력 의존도를 크게 줄이고, 작업 속도와 정확성을 대폭 향상시킬 수 있다.

- 예측 유지보수 및 데이터 기반 관리
  로봇이 수집한 센서 데이터와 환경 정보를 웹으로 실시간 제공하며, AI 분석을 통해 이상 발생 가능성을 사전에 예측합니다. 이를 통해 불필요한 점검・정비 비용을 절감하고, 설비 장애 및 사고를 미연에 방지할 수 있다.

- 원격 제어 및 실시간 진단
  웹과 서버 연동을 통해 관리자(운영자)가 원격에서 로봇 상태를 실시간으로 모니터링・제어할 수 있어, 현장 방문 없이도 즉각적인 대응과 유지보수 계획 수립이 가능합니다.

(2) 시장성 및 가격 우위성

- 운영비 절감 및 ROI 향상
  자동화・예측 유지보수로 인건비, 점검 시간, 설비 중단 비용이 대폭 절감되며, 기존 인력 중심 방식 대비 투자 대비 효율(ROI)이 높습니다.
  실제 덕트 점검/청소 로봇 시장은 인건비 절감, 작업시간 단축, 안전성 확보 등에서 기존 방식 대비 우위로 인해 수요가 빠르게 증가하고 있습니다.

- 확장성 및 다양한 산업 적용
  상업・산업용 건물, 데이터센터, 발전소 등 다양한 분야에 적용 가능하며, 스마트 빌딩・스마트 팩토리 등 자동화 트렌드와 맞물려 시장 성장성이 매우 높습니다.

- 가격 경쟁력
  덕트 점검・청소 로봇은 초기 도입비용이 있으나, 자동화로 인한 인건비 및 유지보수 비용의 지속적 절감 효과로 인해 전체 운영비용에서 가격 우위성을 확보합니다.
  기존 시장에서는 평균적으로 $2,000~$10,000 수준의 로봇이 공급되고 있으며, 이는 반복적인 인력 투입 대비 장기적으로 비용 효율적임이 입증되고 있습니다.

이 프로젝트는 자동화・AI 기반 예측 유지보수, 실시간 데이터 관리 등 첨단 기능을 결합해 기존 서비스 대비 명확한 차별성과 시장 내 가격 우위성을 갖추고 있다. 이러한 기술적・경제적 강점은 관련 시장에서 빠른 확산과 높은 성장성을 기대할 수 있다.

5.2 주요 활용분야

(1) 설비 관리 자동화 및 비용 절감

본 시스템은 SLAM 및 LiDAR 기반 실시간 2D 지도를 작성하여, 덕트 내부의 복잡한 구조를 정밀하게 매핑하고, 자동 데이터 수집 및 이상 감지 기능을 지원한다. 이를 통해 각종 센싱데이터(위치, 환경, 케이블 이상 신호 등)가 실시간으로 취합 및 분석되며, 엔지니어링 유지보수 전략의 최적화와 케이블·전력선 진단 및 점검 자동화가 가능하다. 실제 산업현장에서 적용 시 반복적·위험작업의 자동화로 인력 소모와 현장 방문 빈도, 운영 리스크, 유지관리 비용을 획기적으로 줄일 수 있다.

(2) 안전성 및 환경 관리 강화

덕트 환경에서의 실시간 원격 모니터링, 결함 조기 경보, 위험구간 자동 탐지 등을 구현해 운영 안정성을 극대화한다. 향후 가스감지센서, 미세먼지·환경센서, 비접촉 전류 센서 등 추가 모듈을 탑재하면, 전력선 누전·과부하·과온, 가스누출, 배관 부식·파손, 화재 위험 등의 조기 감지 및 자동 알람·긴급 대응도 가능하다. 이를 통해 현장 환경의 안전 수준을 크게 향상시키는 핵심 유지관리 인프라로 활용될 수 있다.

(3) 데이터 기반의 지능형 유지보수

플랫폼에 연동된 웹 기반 데이터베이스(MongoDB Atlas 등)와 실시간 원격 제어·진단 인터페이스를 통해, 공조·에너지·플랜트 등 다양한 산업 현장에서 상태 데이터를 효과적으로 축적·분석·예측할 수 있다. AI 기반 추론모델을 결합하면 케이블 손상·이상 징후의 조기 예측, 자동 유지보수 일정 수립, 현장 관리자·엔지니어의 의사결정 지원 등 지능형 자산관리가 현실화되는데 활용될 수 있다.

(4) 지속적인 무인 운영 자동화

ROS2 기반 자율주행 알고리즘 및 신뢰성 높은 Mapping 기술을 통해, 시설 내 무인 순찰·점검 시스템, 야간·휴일 무중단 운영 등 완전 무인화 시나리오 구현이 가능하다. 더불어 다중 로봇 동기화, 원격 Batch 제어, 자동 배터리 관리까지 확장되면 대규모 시설의 스마트 자산관리, 재난 예방, 연계 자동화 공정의 핵심 거점으로 기여할 수 있다.

(5) 신규 확장 및 파생 활용처

- 대형 인프라(플랜트, 원전, 열공급망, 데이터센터 등) 내 배관·케이블 동시 진단 및 관리 자동화
- 건설 현장, 에너지/통신사 트렌치 관리, 지하 공공 인프라 유지보수
- 스마트 홈/빌딩의 공조·환기·소방라인 상시 자율진단 및 실내 환경 질 관리
- 녹색건축/ESG 실현을 위한 친환경 설비 진단, 인프라 운영 최적화, 도시 재난·유지관리 대응

Ⅵ. 참 고 문 헌

[1] S. Hou, B. Dong, H. Wang, and G. Wu, “Inspection of surface defects on stay cables using a robot and transfer learning,“ Automation in Construction, vol. 119, no. 103382, pp. 1-14, 2020.

[2] S. Miyake, K. Yoshida, S. Sugano, and M. Kamezaki, “Preliminary design and evaluation of a ducted-fan type pipeline robot,“ ROBOMECH Journal, vol. 11, no. 17, pp. 1-10, 2024.

[3] Priyanto Hidayatullah et al., “YOLOv8 to YOLO11: A Comprehensive Architecture In-depth Comparative Review“, arXiv preprint, 2025.

[4] Santos-González I.,et al “Implementation and Analysis of Real-Time Streaming Protocols”, Sensors, vol. 17, no. 4, article 846, 2017, doi:10.3390/s17040846

[5] W. Shi, J. Cao, Q. Zhang, Y. Li and L. Xu, “Edge Computing: Vision and Challenges,” IEEE Internet of Things Journal, vol. 3, no. 5, pp. 637-646, Oct. 2016, doi: 10.1109/JIOT.2016.2579198.

[6] Guardiani, P., Canali, C., Pistone, A., Leggieri, S., Gloriani, C., Rahman, N., Cannella, F., & Caldwell, D. G. (2018). Novel integrated robotic system for tiny duct inspection. Procedia Manufacturing, 17, 342-349. DOI:10.1016/j.promfg.2018.10.055

[7] Jang, H., Kim, T. Y., Lee, Y. C., Song, Y. H., & Choi, H. R. (2022). Autonomous navigation of in-pipe inspection robot using contact sensor modules. IEEE/ASME Transactions on Mechatronics, 27(6), 4665-4674. https://doi.org/10.1109/TMECH.2022.3162192

[8] Roh, S.-G., & Choi, H. R. (2005). Differential-drive in-pipe robot for moving inside urban gas pipelines. IEEE Transactions on Robotics, 21(1), 1-17. https://doi.org/10.1109/TRO.2004.838000

[9] Rayhana, R., Yun, H., Liu, Z., & Kong, X. (2024). Automated defect-detection system for water pipelines based on CCTV inspection videos of autonomous robotic platforms. IEEE/ASME Transactions on Mechatronics, 29(3), 2021-2031. https://doi.org/10.1109/TMECH.2023.3307594

Ⅶ. 부 록

부품리스트

품 명	규 격	제작사 및 일련번호	단가(원)	구입처 (상호명 또는 인터넷사이트)
Robotis TurtleBot3 Burger	-	(주)로보티즈	786,500	(주)로보티즈
Robotis TB3 Caterpillar Set	궤도형 트랙	(주)로보티즈	16,940	(주)로보티즈
맞춤형 휠	9.0cm직경, PLA	-	-	자체 제작
베어링 및 L자 브래킷	상용 규격품	미스미, (주)로보티즈	10,000	미스미, (주)로보티즈
Raspberry Pi 4	Model B, 4GB	Raspberry Pi Foundation	82,500	디바이스마트
Raspberry Pi 4	Model B, 4GB	Raspberry Pi Foundation	82,500	디바이스마트
Raspberry Pi AI Camera		디바이스마트	95,000	디바이스마트
LDS-02	360도 2D LiDAR	Robotis	100,000	(주)로보티즈
IMU Sensor	-	Robotis	32,000	(주)로보티즈
Dynamic - cell XL-430	-	Robotis	31,900	(주)로보티즈
리튬 폴리머 전지	11.1V 1800mAh	Robotis	65,340	상용 부품
일자형 덕트	폭 : 50cm, 높이 : 1m	-	-	-
사다리형 케이블 트레이	폭 : 2m, 높이 : 2.5m	㈜우리정공	85000	㈜우리정공
Ubuntu	22.04 LTS	Canonical	오픈소스	
ROS2	Humble	Open Robotics	오픈소스	
Nav2	-	Open Robotics	오픈소스	
Python	3.10	Python Software Foundation	오픈소스	

1) 품명에는 H/W소자 및 S/W package 등도 포함됨.
2) 규격에는 전기적 특성과 용도등의 규격을 말함.

Ⅶ. 부 록

품 명	규 격	제작사 및 일련번호	단가(원)	구입처 (상호명 또는 인터넷사이트)
YOLOv11 Nano	-	Ultralytics	오픈소스	
YOLOv8	-	Ultralytics	오픈소스	
MongoDB Atlas	-	MongoDB, Inc.	무료 티어	

1) 품명에는 H/W소자 및 S/W package 등도 포함됨.
2) 규격에는 전기적 특성과 용도등의 규격을 말함.

Ⅷ. 감사의 글

본 작품의 연구는 서울과학기술대학교 전자IT미디어공학과에서 이루어졌으며, 설계부터 완성에 이르기까지 많은 분들의 도움을 받았습니다.
먼저 주제 선정부터 작품의 완성이 이루어지기까지 학문적 지도는 물론 모든 일에 깊은 관심으로 지도해 주신 교수님께 감사의 말씀을 드립니다.
또한 연구실 사용에 있어 양해해 주신 회로설계 연구실의 모든 분들과 한이음 드림업 프로젝트의 재정적 지원에 깊이 감사드립니다.

2025년 11월 3일
고 은 찬
윤 한 준
김 연 재
손 윤 석